

CAHIER DES CHARGES : PROJET

"TR(A)UCK"

Systeme d'optimisation logicielle des trajets, des chauffeurs et
des chargements

Membres de projet :

Salaheddine Manaa

Yassine Janane

Anass Chenguiti

Yassir Amrani

Aymane Ouchchate

Table des matières

1	Présentation Générale	2
2	Spécifications Fonctionnelles	2
2.1	Module Web Central (Administrateurs & Dispatchers)	2
2.2	Module Chauffeur (Application Web Mobile / PWA)	2
3	Architecture Technique et Stack Technologique	2
3.1	Front-end : React.js	2
3.2	Back-end : ASP.NET Core (C#)	3
3.3	Bases de Données : Approche Polyglotte	3
4	Implémentation Algorithmique (Routage et Optimisation)	3
4.1	Algorithme de Dijkstra	3
4.2	Algorithme de Bellman-Ford	3
5	Exigences Non Fonctionnelles	3
6	Étapes de Réalisation et Livrables	4
7	Chronologie de Réalisation (Planning sur 2 mois)	4

1 Présentation Générale

Le projet "Tr(a)uck" est une plateforme logicielle centralisée de gestion logistique. Son objectif est d'optimiser l'affectation des chauffeurs, le calcul des itinéraires et le chargement des camions sans recourir à des capteurs matériels ou des caméras (IoT). L'intelligence du système repose sur le traitement algorithmique avancé des données, l'utilisation des smartphones des chauffeurs comme terminaux interactifs, et une architecture web robuste et réactive.

2 Spécifications Fonctionnelles

2.1 Module Web Central (Administrateurs & Dispatchers)

- **Tableau de bord interactif** : Visualisation globale de la flotte, des commandes en cours et des alertes en temps réel.
- **Planification des chargements (Bin Packing 3D)** : Calcul et prévisualisation 3D de l'agencement optimal des colis dans les remorques en fonction de l'ordre de livraison.
- **Gestion des ressources** : Base de données des chauffeurs (compétences, heures de conduite, historique) et des véhicules (capacités, caractéristiques).
- **Supervision des trajets** : Suivi de la progression des livraisons avec mise à jour des statuts en direct.

2.2 Module Chauffeur (Application Web Mobile / PWA)

- **Feuille de route dynamique** : Affichage de l'itinéraire optimisé et de l'ordre de livraison.
- **Geofencing et Notifications** : Déclenchement automatique d'un statut "En approche" lorsque les coordonnées GPS du smartphone entrent dans un rayon prédéfini du point de livraison.
- **Bouton SOS / Signalement** : Interface à un clic permettant au chauffeur de signaler une panne, un retard ou un accident, déclenchant un recalcul immédiat au niveau du serveur.
- **Preuve de livraison** : Signature électronique ou validation de la livraison directement sur le smartphone.

3 Architecture Technique et Stack Technologique

Pour assurer la performance, la scalabilité et la séparation des préoccupations, le projet s'articule autour de l'architecture suivante :

3.1 Front-end : React.js

- **Rôle** : Interface utilisateur dynamique (SPA - Single Page Application) pour les gestionnaires et vue responsive pour les chauffeurs.
- **Bibliothèques clés** :
 - Redux ou Zustand pour la gestion d'état complexe.

- React-Leaflet ou API Mapbox GL JS pour l'intégration cartographique interactive.
- Three.js (optionnel) pour la prévisualisation 3D du chargement.

3.2 Back-end : ASP.NET Core (C#)

- **Rôle** : Cœur logique du système. Gestion des requêtes sécurisées, exécution des algorithmes d'optimisation.
- **Caractéristiques** : Architecture en API RESTful et utilisation de SignalR pour la communication bidirectionnelle en temps réel.

3.3 Bases de Données : Approche Polyglotte

- **PostgreSQL (Base Relationnelle)** : Stockage des données structurées et critiques exigeant l'intégrité (ACID). Extension PostGIS indispensable pour stocker des coordonnées géographiques et effectuer des requêtes spatiales complexes.
- **NoSQL - MongoDB ou Redis** : Gestion des flux de données à haute fréquence. Utilisé pour stocker les logs de télémétrie GPS envoyés par les smartphones des chauffeurs.

4 Implémentation Algorithmique (Routage et Optimisation)

L'intelligence de routage côté ASP.NET s'appuiera sur des structures de graphes pour modéliser le réseau routier.

4.1 Algorithme de Dijkstra

- **Cas d'usage principal** : Planification initiale des trajets.
- **Fonctionnement** : Utilisé pour trouver le chemin le plus court et le plus rapide entre l'entrepôt et les points de livraison dans des conditions normales (poids des arêtes strictement positifs).

4.2 Algorithme de Bellman-Ford

- **Cas d'usage principal** : Recalcul dynamique avec contraintes complexes ou pénalités (Routing d'urgence).
- **Fonctionnement** : Capable de gérer des graphes où certaines arêtes peuvent avoir des "poids négatifs" ou des pénalités sévères.
- **Application** : En cas d'incident, le système attribue une pénalité extrême à certains tronçons pour redistribuer un trajet en évitant les zones à risques.

5 Exigences Non Fonctionnelles

- **Sécurité** : Authentification par JWT (JSON WebTokens) entre React et ASP.NET. Chiffrement des données sensibles.

- **Haute Disponibilité & Scalabilité** : Architecture conteneurisée via Docker pour isoler les services (React, ASP.NET, PostgreSQL, NoSQL).
- **Tolérance aux pannes** : Mode hors-ligne pour la PWA du chauffeur grâce aux Service Workers.

6 Étapes de Réalisation et Livrables

1. **Phase 1** : Modélisation et Maquettage. Schéma de BDD et UI/UX pour React.
2. **Phase 2** : Développement Back-end. Création API ASP.NET, SignalR, algorithmes Dijkstra/Bellman-Ford.
3. **Phase 3** : Développement Front-end. Vues React, cartographie, PWA mobile.
4. **Phase 4** : Tests et Intégration. Test de charge, simulation flux GPS, vérification Bin Packing.
5. **Phase 5** : Déploiement. Conteneurisation (Docker) et mise en production.

7 Chronologie de Réalisation (Planning sur 2 mois)

Mois 1 : Fondations, Back-end et Algorithmique

- **Semaine 1 : Conception, Architecture et Setup.** Définition de l'architecture, maquettes UI/UX, modélisation BDD (PostGIS/NoSQL), initialisation des dépôts.
- **Semaine 2 : Développement du cœur Back-end.** API RESTful (ASP.NET Core), sécurité JWT, module CRUD pour la gestion de la flotte.
- **Semaine 3 : Algorithmes de Routage et Logique Métier.** Algorithme de Bin Packing 3D, implémentation de Dijkstra (trajets initiaux) et de Bellman-Ford (gestion d'urgence).
- **Semaine 4 : Base Front-end (React) et Temps Réel.** Setup React, state management (Redux/Zustand), composants de base, implémentation de SignalR.

Mois 2 : Front-end Avancé, Intégration et Déploiement

- **Semaine 5 : Intégration Cartographique et App Chauffeur.** Carte interactive avec React-Leaflet/Mapbox, développement PWA mobile, bouton SOS.
- **Semaine 6 : Flux GPS, NoSQL et Mode Hors-ligne.** Connexion PWA vers NoSQL via ASP.NET, mise à jour en direct via SignalR, développement Service Workers pour le mode hors-ligne.
- **Semaine 7 : Tests, Débogage et Recettage.** Tests unitaires, tests d'intégration, tests de charge (SignalR/NoSQL), correction de bugs.
- **Semaine 8 : Conteneurisation et Déploiement.** Fichiers Dockerfile et docker-compose.yml, déploiement Cloud, documentation et formation.